

OASIS v3 Termux Orchestrator Scripts

Ultra-Lightweight AI Command Center (<5MB Footprint)



Executive Summary

This document provides comprehensive Termux orchestrator scripts for OASIS v3 (Open Source AI Superintelligence Ecosystem) implementing a revolutionary ultra-lightweight architecture. The system utilizes Termux as a <5MB command center to orchestrate HuggingFace Spaces for all heavy AI processing, eliminating dependency hell while maintaining full control from mobile devices.

Key Revolutionary Features:

- Ultra-lightweight Python architecture (urllib only, zero ML dependencies)
- 100% API-first approach with HuggingFace Spaces integration
- Real-time business logic controller and revenue optimization
- Comprehensive monitoring and deployment management
- Mobile-optimized performance with <5MB total footprint



1. Main Orchestrator Script

File: termux_orchestrator.py

```
#!/usr/bin/env python3
"""
OASIS v3 Termux Orchestrator - Ultra-Lightweight Command Center
Revolutionary architecture: Termux (<5MB) + HuggingFace Spaces supremacy
Zero heavy dependencies - Pure Python + urllib approach
"""

import json
import urllib.request
import urllib.parse
import urllib.error
import time
import sys
import os
from datetime import datetime, timedelta
import threading
import queue
import hashlib
import base64

class OasisOrchestrator:
    """Ultra-lightweight OASIS v3 command center"""

    def __init__(self, config_path="oasis_config.json"):
```

```

self.config = self._load_config(config_path)
self.session_id = self._generate_session_id()
self.metrics = {
    "requests_sent": 0,
    "responses_received": 0,
    "errors": 0,
    "total_processing_time": 0,
    "revenue_generated": 0.0,
    "start_time": datetime.now()
}
self.request_queue = queue.Queue()
self.response_cache = {}
self.health_status = {"status": "initializing", "last_check": None}
print(f"🚀 OASIS v3 Orchestrator initialized - Session: {self.session_id}")

def _load_config(self, config_path):
    """Load configuration with fallback defaults"""
    default_config = {
        "hf_spaces_url": "https://your-oasis-space.hf.space",
        "api_key": "",
        "max_retries": 3,
        "timeout": 30,
        "cache_enabled": True,
        "monitoring_interval": 60,
        "revenue_target_daily": 166.67, # $50K/month = $166.67/day
        "batch_size": 10,
        "compression_enabled": True
    }

    try:
        if os.path.exists(config_path):
            with open(config_path, 'r') as f:
                user_config = json.load(f)
                default_config.update(user_config)
            return default_config
    except Exception as e:
        print(f"⚠️ Config load error: {e}, using defaults")
        return default_config

def _generate_session_id(self):
    """Generate unique session identifier"""
    timestamp = str(int(time.time()))
    random_data = os.urandom(8).hex()
    return f"oasis-_{timestamp}_{random_data[:8]}"

def _make_request(self, endpoint, data=None, method="GET"):
    """Ultra-lightweight HTTP request handler"""
    url = f"{self.config['hf_spaces_url']}/{endpoint.lstrip('/')}"

    try:
        # Prepare request
        headers = {
            'User-Agent': f'OASIS-v3-Termux/{self.session_id}',
            'Content-Type': 'application/json',
            'Accept': 'application/json',
            'X-Session-ID': self.session_id
        }

        if self.config.get('api_key'):
            headers['Authorization'] = f"Bearer {self.config['api_key']}"

        # Handle request data
        request_data = None
        if data and method == "POST":
            request_data = json.dumps(data).encode('utf-8')
            headers['Content-Length'] = str(len(request_data))

        # Create request
        req = urllib.request.Request(url, data=request_data, headers=headers, meth

        # Execute with timeout
        start_time = time.time()
        with urllib.request.urlopen(req, timeout=self.config['timeout']) as response:
            response_data = response.read().decode('utf-8')
            processing_time = time.time() - start_time

        # Update metrics
        self.metrics["requests_sent"] += 1
        self.metrics["responses_received"] += 1
        self.metrics["total_processing_time"] += processing_time

```

```

        return {
            "success": True,
            "data": json.loads(response_data) if response_data else {},
            "status_code": response.status,
            "processing_time": processing_time
        }

    except urllib.error.HTTPError as e:
        self.metrics["errors"] += 1
        error_data = e.read().decode('utf-8') if e.fp else "No error details"
        return {
            "success": False,
            "error": f"HTTP {e.code}: {e.reason}",
            "details": error_data,
            "status_code": e.code
        }

    except urllib.error.URLError as e:
        self.metrics["errors"] += 1
        return {
            "success": False,
            "error": f"Connection error: {e.reason}",
            "details": str(e)
        }

    except Exception as e:
        self.metrics["errors"] += 1
        return {
            "success": False,
            "error": f"Request failed: {str(e)}",
            "details": type(e).__name__
        }

def process_ai_request(self, operation, payload):
    """Process AI operation through HuggingFace Spaces"""
    endpoint = f"api/v3/ai/{operation}"

    # Add session metadata
    payload.update({
        "session_id": self.session_id,
        "timestamp": datetime.now().isoformat(),
        "source": "termux_orchestrator"
    })

    print(f"🧠 Processing AI request: {operation}")
    result = self._make_request(endpoint, payload, "POST")

    if result["success"]:
        print(f"✅ AI processing completed in {result['processing_time']:.2f}s")
        # Update revenue metrics
        self._update_revenue_metrics(operation, result["data"])
    else:
        print(f"❌ AI processing failed: {result['error']}")

    return result

def batch_process(self, requests_batch):
    """Process multiple requests efficiently"""
    endpoint = "api/v3/batch"

    batch_payload = {
        "requests": requests_batch,
        "session_id": self.session_id,
        "batch_size": len(requests_batch),
        "compression": self.config.get("compression_enabled", True)
    }

    print(f"📦 Processing batch of {len(requests_batch)} requests")
    result = self._make_request(endpoint, batch_payload, "POST")

    if result["success"]:
        print(f"✅ Batch processed successfully")
        # Process individual results
        for i, item_result in enumerate(result["data"].get("results", [])):
            if item_result.get("success"):
                self._update_revenue_metrics(requests_batch[i].get("operation"), i)

    return result

```

```

def get_system_status(self):
    """Get comprehensive system status"""
    endpoint = "api/v3/status"
    result = self._make_request(endpoint)

    if result["success"]:
        # Combine with local metrics
        status = result["data"]
        status["termux_metrics"] = self.metrics
        status["session_id"] = self.session_id
        status["local_health"] = self.health_status
        return status

    return {"error": "Failed to get system status", "details": result}

def monitor_performance(self):
    """Continuous performance monitoring"""
    while True:
        try:
            # Check system health
            status = self.get_system_status()

            if isinstance(status, dict) and not status.get("error"):
                self.health_status = {
                    "status": "healthy",
                    "last_check": datetime.now().isoformat(),
                    "metrics": self.metrics,
                    "uptime": str(datetime.now() - self.metrics["start_time"])
                }

                # Calculate performance metrics
                success_rate = (self.metrics["responses_received"] /
                                max(self.metrics["requests_sent"], 1)) * 100
                avg_response_time = (self.metrics["total_processing_time"] /
                                     max(self.metrics["responses_received"], 1))

                print(f"🇩🇪 Performance: {success_rate:.1f}% success, "
                      f"{avg_response_time:.2f}s avg response, "
                      f"${self.metrics['revenue_generated']:.2f} revenue")
            else:
                self.health_status["status"] = "unhealthy"
                print("⚠️ System health check failed")

            time.sleep(self.config["monitoring_interval"])

        except KeyboardInterrupt:
            print("🛑 Monitoring stopped by user")
            break
        except Exception as e:
            print(f"❌ Monitoring error: {e}")
            time.sleep(30) # Wait before retry

def _update_revenue_metrics(self, operation, result_data):
    """Update revenue tracking"""
    # Revenue calculation based on operation type
    revenue_map = {
        "text_generation": 0.01,
        "image_processing": 0.02,
        "sentiment_analysis": 0.005,
        "summarization": 0.015,
        "translation": 0.01,
        "question_answering": 0.008
    }

    revenue = revenue_map.get(operation, 0.01)
    self.metrics["revenue_generated"] += revenue

def start_monitoring_thread(self):
    """Start background monitoring"""
    monitor_thread = threading.Thread(target=self.monitor_performance, daemon=True)
    monitor_thread.start()
    print("🕒 Background monitoring started")

def get_revenue_report(self):
    """Generate revenue analytics report"""
    uptime_hours = (datetime.now() - self.metrics["start_time"]).total_seconds() /
    hourly_revenue = self.metrics["revenue_generated"] / max(uptime_hours, 1)
    daily_projection = hourly_revenue * 24

```

```

monthly_projection = daily_projection * 30

return {
    "current_revenue": self.metrics["revenue_generated"],
    "hourly_rate": hourly_revenue,
    "daily_projection": daily_projection,
    "monthly_projection": monthly_projection,
    "target_progress": (daily_projection / self.config["revenue_target_daily"])
    "uptime_hours": uptime_hours
}

# Command-line interface
def main():
    """Main orchestrator interface"""
    print("🚀 OASIS v3 Termux Orchestrator - Ultra-Lightweight Command Center")
    print("Revolutionary AI orchestration with <5MB footprint")
    print("-" * 60)

    orchestrator = OasisOrchestrator()
    orchestrator.start_monitoring_thread()

    while True:
        try:
            print("\n📁 Available Operations:")
            print("1. Text Generation")
            print("2. Image Processing")
            print("3. Sentiment Analysis")
            print("4. System Status")
            print("5. Revenue Report")
            print("6. Batch Processing")
            print("0. Exit")

            choice = input("\n🎯 Select operation (0-6): ").strip()

            if choice == "0":
                print("👋 OASIS v3 Orchestrator shutdown")
                break
            elif choice == "1":
                text = input("Enter text to generate: ")
                result = orchestrator.process_ai_request("text_generation", {"text": t
                print(f"📝 Result: {result}")
            elif choice == "2":
                image_url = input("Enter image URL: ")
                result = orchestrator.process_ai_request("image_processing", {"image_u
                print(f"🖼️ Result: {result}")
            elif choice == "3":
                text = input("Enter text for sentiment analysis: ")
                result = orchestrator.process_ai_request("sentiment_analysis", {"text"
                print(f"😊 Result: {result}")
            elif choice == "4":
                status = orchestrator.get_system_status()
                print(f"📊 System Status: {json.dumps(status, indent=2)}")
            elif choice == "5":
                report = orchestrator.get_revenue_report()
                print(f"💰 Revenue Report: {json.dumps(report, indent=2)}")
            elif choice == "6":
                print("🔄 Batch processing mode - Enter requests (empty line to finish)
                batch_requests = []
                while True:
                    req = input("Request (operation:payload): ").strip()
                    if not req:
                        break
                    try:
                        operation, payload = req.split(":", 1)
                        batch_requests.append({"operation": operation, "payload": json
                    except:
                        print("❌ Invalid format. Use: operation:{'key':'value'}")

                if batch_requests:
                    result = orchestrator.batch_process(batch_requests)
                    print(f"📦 Batch Result: {result}")
                else:
                    print("❌ Invalid choice")

        except KeyboardInterrupt:
            print("\n👋 OASIS v3 Orchestrator shutdown")
            break
        except Exception as e:
            print(f"❌ Error: {e}")

```

```
if __name__ == "__main__":
    main()
```



2. Setup Script

File: setup_termux.sh

```
#!/bin/bash
# OASIS v3 Termux Setup Script - Ultra-Lightweight Installation
# Revolutionary architecture: <5MB footprint, zero heavy dependencies

echo "🚀 OASIS v3 Termux Setup - Ultra-Lightweight AI Command Center"
echo "===== "

# Update Termux packages
echo "📦 Updating Termux packages..."
pkg update -y
pkg upgrade -y

# Install minimal Python environment (only essential packages)
echo "🐍 Installing minimal Python environment..."
pkg install -y python
pkg install -y python-pip

# Verify Python installation
echo "✅ Verifying Python installation..."
python --version
pip --version

# Create OASIS directory structure
echo "📁 Creating OASIS v3 directory structure..."
mkdir -p ~/oasis-v3
cd ~/oasis-v3

# Create subdirectories
mkdir -p {config,logs,cache,scripts,docs}

# Install ONLY essential dependencies (urllib is built-in)
echo "📦 Installing minimal dependencies..."
cat > requirements_termux.txt << EOF
# OASIS v3 Termux Requirements - Ultra-Lightweight
# Total size: <1MB dependencies
# Zero ML libraries - Pure API approach

# Core utilities (built-in with Python)
# urllib.request - HTTP client (built-in)
# json - JSON processing (built-in)
# threading - Concurrent processing (built-in)
# queue - Queue management (built-in)
# hashlib - Cryptographic hashing (built-in)
# base64 - Encoding utilities (built-in)
# datetime - Time handling (built-in)
# os - Operating system interface (built-in)
# sys - System-specific parameters (built-in)
# time - Time-related functions (built-in)

# Optional lightweight additions (if needed)
# No external dependencies required!
# Pure Python + stdlib approach
EOF

echo "✅ Requirements file created with zero external dependencies"

# Download OASIS v3 scripts
echo "📄 Setting up OASIS v3 orchestrator scripts..."

# Create configuration file
cat > config/oasis_config.json << 'EOF'
{
    "hf_spaces_url": "https://your-oasis-space.hf.space",
    "api_key": "",
```

```

    "max_retries": 3,
    "timeout": 30,
    "cache_enabled": true,
    "monitoring_interval": 60,
    "revenue_target_daily": 166.67,
    "batch_size": 10,
    "compression_enabled": true,
    "mobile_optimization": {
        "low_bandwidth_mode": true,
        "request_compression": true,
        "response_caching": true,
        "batch_requests": true
    },
    "business_settings": {
        "pricing_model": "usage_based",
        "revenue_tracking": true,
        "analytics_enabled": true,
        "performance_optimization": true
    }
}
EOF

# Create launch script
cat > scripts/start_oasis.sh << 'EOF'
#!/bin/bash
# OASIS v3 Quick Start Script

echo "🚀 Starting OASIS v3 Termux Orchestrator..."
cd ~/oasis-v3

# Check if configuration exists
if [ ! -f "config/oasis_config.json" ]; then
    echo "❌ Configuration not found. Please run setup first."
    exit 1
fi

# Start orchestrator
python termux_orchestrator.py
EOF

chmod +x scripts/start_oasis.sh

# Create monitoring script
cat > scripts/monitor_oasis.sh << 'EOF'
#!/bin/bash
# OASIS v3 System Monitor

echo "📊 OASIS v3 System Monitor"
echo "===== "

# Display system resources
echo "💻 Memory Usage:"
free -h

echo "💾 Storage Usage:"
df -h $HOME

echo "🔋 Battery Status (if available):"
if command -v termux-battery-status &> /dev/null; then
    termux-battery-status
fi

echo "🌐 Network Status:"
ping -c 3 google.com

echo "🚀 OASIS Process Status:"
ps aux | grep oasis || echo "No OASIS processes running"
EOF

chmod +x scripts/monitor_oasis.sh

# Create backup script
cat > scripts/backup_oasis.sh << 'EOF'
#!/bin/bash
# OASIS v3 Configuration Backup

BACKUP_DIR="$HOME/oasis-backup"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)

```

```

echo "📁 Creating OASIS v3 backup..."
mkdir -p "$BACKUP_DIR"

# Backup configuration and logs
tar -czf "$BACKUP_DIR/oasis_backup_${TIMESTAMP}.tar.gz" \
    config/ logs/ scripts/ 2>/dev/null

echo "✅ Backup created: oasis_backup_${TIMESTAMP}.tar.gz"
echo "📁 Location: $BACKUP_DIR"
EOF

chmod +x scripts/backup_oasis.sh

# Create documentation
cat > docs/QUICKSTART.md << 'EOF'
# OASIS v3 Termux Quick Start Guide

## 🚀 Ultra-Lightweight AI Command Center

### Installation Complete!
Your OASIS v3 system is now installed with <5MB footprint.

### Quick Commands:
```bash
Start OASIS orchestrator
~/oasis-v3/scripts/start_oasis.sh

Monitor system
~/oasis-v3/scripts/monitor_oasis.sh

Backup configuration
~/oasis-v3/scripts/backup_oasis.sh
```

### Configuration:
Edit `config/oasis_config.json` to customize settings.

### Zero Dependencies:
This system uses only Python stdlib - no external packages required!
EOF

# Set up environment variables
echo "🔧 Setting up environment variables..."
echo 'export OASIS_HOME="$HOME/oasis-v3"' >> ~/.bashrc
echo 'export PATH="$OASIS_HOME/scripts:$PATH"' >> ~/.bashrc

# Final setup steps
echo "🔍 Performing system checks..."

# Check available space
echo "💾 Storage Check:"
df -h $HOME | tail -1

# Check Python modules
echo "🐍 Python Module Check:"
python -c "import urllib.request, json, threading, queue, hashlib, base64, datetime, c

# Create initial log file
touch logs/oasis_orchestrator.log
echo "$(date): OASIS v3 setup completed" >> logs/oasis_orchestrator.log

echo ""
echo "🎉 OASIS v3 Setup Complete!"
echo "=====
echo "📁 Installation directory: ~/oasis-v3"
echo "📁 Total footprint: <5MB"
echo "📁 Zero heavy dependencies"
echo "🚀 Ready for AI orchestration"
echo ""
echo "📺 Start with: ~/oasis-v3/scripts/start_oasis.sh"
echo "📖 Documentation: ~/oasis-v3/docs/QUICKSTART.md"
echo ""
echo "🌟 Welcome to the future of mobile AI orchestration!"

```



3. Configuration Manager

File: oasis_config.py

```
#!/usr/bin/env python3
"""
OASIS v3 Ultra-Lightweight Configuration Manager
Zero dependencies - Pure Python implementation
"""

import json
import os
import sys
from datetime import datetime

class OasisConfig:
    """Ultra-lightweight configuration management"""

    def __init__(self, config_file="config/oasis_config.json"):
        self.config_file = config_file
        self.config_data = {}
        self.load_config()

    def load_config(self):
        """Load configuration with error handling"""
        try:
            if os.path.exists(self.config_file):
                with open(self.config_file, 'r') as f:
                    self.config_data = json.load(f)
                print(f"✅ Configuration loaded from {self.config_file}")
            else:
                self.create_default_config()
                print(f"📄 Default configuration created at {self.config_file}")
        except Exception as e:
            print(f"❌ Config load error: {e}")
            self.create_default_config()

    def create_default_config(self):
        """Create default configuration"""
        self.config_data = {
            "version": "3.0.0",
            "hf_spaces": {
                "url": "https://your-oasis-space.hf.space",
                "api_key": "",
                "timeout": 30,
                "max_retries": 3
            },
            "termux": {
                "max_memory_mb": 512,
                "cache_size_mb": 50,
                "log_level": "INFO",
                "monitoring_interval": 60
            },
            "business": {
                "revenue_target_daily": 166.67,
                "pricing_model": "usage_based",
                "analytics_enabled": true,
                "performance_tracking": true
            },
            "optimization": {
                "batch_processing": true,
                "request_compression": true,
                "response_caching": true,
                "low_bandwidth_mode": true
            },
            "security": {
                "session_timeout": 3600,
                "api_rate_limit": 100,
                "encryption_enabled": false
            }
        }
        self.save_config()

    def save_config(self):
        """Save configuration to file"""
        try:
            # Ensure directory exists
            os.makedirs(os.path.dirname(self.config_file), exist_ok=True)
```

```

        with open(self.config_file, 'w') as f:
            json.dump(self.config_data, f, indent=4)
        print(f"📁 Configuration saved to {self.config_file}")
    except Exception as e:
        print(f"❌ Config save error: {e}")

def get(self, key_path, default=None):
    """Get configuration value using dot notation"""
    keys = key_path.split('.')
    value = self.config_data

    try:
        for key in keys:
            value = value[key]
        return value
    except (KeyError, TypeError):
        return default

def set(self, key_path, value):
    """Set configuration value using dot notation"""
    keys = key_path.split('.')
    config = self.config_data

    # Navigate to parent
    for key in keys[:-1]:
        if key not in config:
            config[key] = {}
        config = config[key]

    # Set final value
    config[keys[-1]] = value
    self.save_config()

def validate_config(self):
    """Validate configuration settings"""
    required_sections = ['hf_spaces', 'termux', 'business', 'optimization']
    issues = []

    for section in required_sections:
        if section not in self.config_data:
            issues.append(f"Missing section: {section}")

    # Validate HF Spaces URL
    hf_url = self.get('hf_spaces.url', '')
    if not hf_url or not hf_url.startswith('https://'):
        issues.append("Invalid HuggingFace Spaces URL")

    # Validate numeric values
    timeout = self.get('hf_spaces.timeout', 0)
    if timeout <= 0 or timeout > 300:
        issues.append("Invalid timeout value (must be 1-300 seconds)")

    revenue_target = self.get('business.revenue_target_daily', 0)
    if revenue_target <= 0:
        issues.append("Invalid revenue target")

    return issues

def get_summary(self):
    """Get configuration summary"""
    return {
        "config_file": self.config_file,
        "version": self.get('version', 'unknown'),
        "hf_spaces_configured": bool(self.get('hf_spaces.api_key')),
        "optimization_enabled": self.get('optimization.batch_processing', False),
        "business_tracking": self.get('business.analytics_enabled', False),
        "last_modified": datetime.fromtimestamp(
            os.path.getmtime(self.config_file)
        ).isoformat() if os.path.exists(self.config_file) else "never"
    }

def main():
    """Configuration management CLI"""
    import sys

    if len(sys.argv) < 2:
        print("Usage: python oasis_config.py [get|set|validate|summary] [args...]")
        return

    config = OasisConfig()

```

```

command = sys.argv[1]

if command == "get":
    if len(sys.argv) != 3:
        print("Usage: python oasis_config.py get key.path")
        return
    value = config.get(sys.argv[2])
    print(f"{sys.argv[2]}: {value}")

elif command == "set":
    if len(sys.argv) != 4:
        print("Usage: python oasis_config.py set key.path value")
        return
    try:
        # Try to parse as JSON first
        value = json.loads(sys.argv[3])
    except:
        # Use as string
        value = sys.argv[3]
    config.set(sys.argv[2], value)
    print(f"✅ Set {sys.argv[2]} = {value}")

elif command == "validate":
    issues = config.validate_config()
    if issues:
        print("❌ Configuration Issues:")
        for issue in issues:
            print(f"  - {issue}")
    else:
        print("✅ Configuration is valid")

elif command == "summary":
    summary = config.get_summary()
    print("📄 Configuration Summary:")
    for key, value in summary.items():
        print(f"  {key}: {value}")

else:
    print(f"❌ Unknown command: {command}")

if __name__ == "__main__":
    main()

```

4. HuggingFace API Client

File: hf_api_client.py

```

#!/usr/bin/env python3
"""
OASIS v3 HuggingFace Spaces API Client
Ultra-lightweight implementation using only urllib
Zero external dependencies - Pure Python approach
"""

import urllib.request
import urllib.parse
import urllib.error
import json
import time
import hashlib
import base64
from datetime import datetime

class HuggingFaceAPIClient:
    """Ultra-lightweight HF Spaces API client"""

    def __init__(self, spaces_url, api_key="", timeout=30):
        self.spaces_url = spaces_url.rstrip('/')
        self.api_key = api_key
        self.timeout = timeout
        self.session_headers = {
            'User-Agent': 'OASIS-v3-Client/1.0',
            'Content-Type': 'application/json',

```

```

        'Accept': 'application/json'
    }

    if api_key:
        self.session_headers['Authorization'] = f'Bearer {api_key}'

    # Request tracking
    self.request_count = 0
    self.total_response_time = 0
    self.error_count = 0

def _make_request(self, endpoint, data=None, method="GET", headers=None):
    """Core HTTP request method"""
    url = f"{self.spaces_url}/{endpoint.lstrip('/')}"

    # Prepare headers
    request_headers = self.session_headers.copy()
    if headers:
        request_headers.update(headers)

    # Prepare data
    request_data = None
    if data and method in ["POST", "PUT", "PATCH"]:
        if isinstance(data, dict):
            request_data = json.dumps(data).encode('utf-8')
        else:
            request_data = str(data).encode('utf-8')

    try:
        # Create and execute request
        req = urllib.request.Request(
            url,
            data=request_data,
            headers=request_headers,
            method=method
        )

        start_time = time.time()
        self.request_count += 1

        with urllib.request.urlopen(req, timeout=self.timeout) as response:
            response_time = time.time() - start_time
            self.total_response_time += response_time

            response_data = response.read().decode('utf-8')

            return {
                "success": True,
                "data": json.loads(response_data) if response_data else {},
                "status_code": response.status,
                "response_time": response_time,
                "headers": dict(response.headers)
            }

    except urllib.error.HTTPError as e:
        self.error_count += 1
        error_body = e.read().decode('utf-8') if e.fp else ""
        return {
            "success": False,
            "error": "HTTP_ERROR",
            "status_code": e.code,
            "message": e.reason,
            "details": error_body
        }

    except urllib.error.URLError as e:
        self.error_count += 1
        return {
            "success": False,
            "error": "CONNECTION_ERROR",
            "message": str(e.reason),
            "details": str(e)
        }

    except Exception as e:
        self.error_count += 1
        return {
            "success": False,
            "error": "REQUEST_ERROR",
            "message": str(e),

```

```

        "details": type(e).__name__
    }

def health_check(self):
    """Check HF Spaces health"""
    return self._make_request("health", method="GET")

def process_text(self, operation, text, **kwargs):
    """Process text through AI models"""
    payload = {
        "operation": operation,
        "text": text,
        "timestamp": datetime.now().isoformat(),
        **kwargs
    }
    return self._make_request("api/v3/ai/text", payload, "POST")

def process_image(self, operation, image_data, **kwargs):
    """Process image through AI models"""
    payload = {
        "operation": operation,
        "image_data": image_data,
        "timestamp": datetime.now().isoformat(),
        **kwargs
    }
    return self._make_request("api/v3/ai/image", payload, "POST")

def batch_process(self, requests_list):
    """Process multiple requests in batch"""
    payload = {
        "requests": requests_list,
        "batch_size": len(requests_list),
        "timestamp": datetime.now().isoformat()
    }
    return self._make_request("api/v3/batch", payload, "POST")

def get_models_info(self):
    """Get available AI models information"""
    return self._make_request("api/v3/models", method="GET")

def get_usage_stats(self):
    """Get usage statistics"""
    return self._make_request("api/v3/stats", method="GET")

def get_business_metrics(self):
    """Get business metrics and revenue data"""
    return self._make_request("api/v3/business/metrics", method="GET")

def submit_feedback(self, request_id, rating, feedback=""):
    """Submit feedback for request quality"""
    payload = {
        "request_id": request_id,
        "rating": rating,
        "feedback": feedback,
        "timestamp": datetime.now().isoformat()
    }
    return self._make_request("api/v3/feedback", payload, "POST")

def get_client_stats(self):
    """Get client-side statistics"""
    avg_response_time = (
        self.total_response_time / max(self.request_count, 1)
    ) if self.request_count > 0 else 0.0

    success_rate = (
        (self.request_count - self.error_count) / max(self.request_count, 1) * 100
    ) if self.request_count > 0 else 0.0

    return {
        "total_requests": self.request_count,
        "successful_requests": self.request_count - self.error_count,
        "error_count": self.error_count,
        "success_rate_percent": success_rate,
        "average_response_time": avg_response_time,
        "total_response_time": self.total_response_time
    }

def test_connection(self):
    """Test connection to HF Spaces"""
    print(f"🔍 Testing connection to {self.spaces_url}")

```

```

result = self.health_check()

if result["success"]:
    print(f"✅ Connection successful (HTTP {result['status_code']})")
    print(f"🕒 Response time: {result['response_time']:.3f}s")
    return True
else:
    print(f"❌ Connection failed: {result.get('message', 'Unknown error')}")
    if result.get("status_code"):
        print(f"🔍 Status Code: {result['status_code']}")
    return False

def interactive_test(self):
    """Interactive testing interface"""
    print("🚀 HuggingFace API Client Interactive Test")
    print("=" * 45)

    while True:
        print("\n📋 Available Operations:")
        print("1. Health Check")
        print("2. Text Processing Test")
        print("3. Get Models Info")
        print("4. Get Usage Stats")
        print("5. Client Statistics")
        print("0. Exit")

        choice = input("\n👉 Select operation (0-5): ").strip()

        if choice == "0":
            break
        elif choice == "1":
            result = self.health_check()
            print(f"🏥 Health Check: {json.dumps(result, indent=2)}")
        elif choice == "2":
            text = input("Enter text to process: ")
            operation = input("Enter operation (default: sentiment): ") or "sentiment"
            result = self.process_text(operation, text)
            print(f"✍️ Text Processing: {json.dumps(result, indent=2)}")
        elif choice == "3":
            result = self.get_models_info()
            print(f"🧠 Models Info: {json.dumps(result, indent=2)}")
        elif choice == "4":
            result = self.get_usage_stats()
            print(f"📊 Usage Stats: {json.dumps(result, indent=2)}")
        elif choice == "5":
            stats = self.get_client_stats()
            print(f"📈 Client Stats: {json.dumps(stats, indent=2)}")
        else:
            print("❌ Invalid choice")

def main():
    """CLI interface for API client"""
    import sys

    if len(sys.argv) < 2:
        print("Usage: python hf_api_client.py <spaces_url> [api_key]")
        print("Example: python hf_api_client.py https://your-space.hf.space your_api_key")
        return

    spaces_url = sys.argv[1]
    api_key = sys.argv[2] if len(sys.argv) > 2 else ""

    client = HuggingFaceAPIClient(spaces_url, api_key)

    # Test connection
    if client.test_connection():
        client.interactive_test()
    else:
        print("❌ Cannot establish connection. Please check your URL and API key.")

if __name__ == "__main__":
    main()

```

```

#!/usr/bin/env python3
"""
OASIS v3 Business Logic Controller
Ultra-lightweight revenue optimization and analytics
Pure Python implementation - Zero external dependencies
"""

import json
import time
import os
from datetime import datetime, timedelta
import urllib.request
import urllib.parse

class BusinessController:
    """Ultra-lightweight business logic and revenue optimization"""

    def __init__(self, config_path="config/oasis_config.json"):
        self.config = self._load_config(config_path)
        self.revenue_data = {
            "daily_revenue": 0.0,
            "monthly_revenue": 0.0,
            "total_requests": 0,
            "successful_requests": 0,
            "start_date": datetime.now().date().isoformat(),
            "last_updated": datetime.now().isoformat()
        }
        self.pricing_model = self._initialize_pricing()
        self.analytics_cache = {}
        self._load_revenue_data()

    def _load_config(self, config_path):
        """Load business configuration"""
        default_config = {
            "revenue_target_daily": 166.67, # $50K/month target
            "pricing_model": "usage_based",
            "analytics_enabled": True,
            "performance_tracking": True,
            "optimization_enabled": True
        }

        try:
            if os.path.exists(config_path):
                with open(config_path, 'r') as f:
                    config_data = json.load(f)
                    return config_data.get("business", default_config)
            return default_config
        except Exception as e:
            print(f"⚠️ Config load error: {e}")
            return default_config

    def _initialize_pricing(self):
        """Initialize dynamic pricing model"""
        return {
            "text_generation": {"base": 0.01, "premium": 0.02},
            "image_processing": {"base": 0.03, "premium": 0.05},
            "sentiment_analysis": {"base": 0.005, "premium": 0.01},
            "summarization": {"base": 0.015, "premium": 0.025},
            "translation": {"base": 0.01, "premium": 0.02},
            "question_answering": {"base": 0.008, "premium": 0.015},
            "batch_processing": {"discount": 0.1, # 10% batch discount
            "premium_multiplier": 1.5,
            "peak_hours_multiplier": 1.2 # 20% increase during peak hours
        }

    def _load_revenue_data(self):
        """Load existing revenue data"""
        revenue_file = "logs/revenue_data.json"
        try:
            if os.path.exists(revenue_file):
                with open(revenue_file, 'r') as f:
                    stored_data = json.load(f)
                    self.revenue_data.update(stored_data)
        except Exception as e:
            print(f"⚠️ Revenue data load error: {e}")

```

```

def _save_revenue_data(self):
    """Save revenue data to file"""
    revenue_file = "logs/revenue_data.json"
    try:
        os.makedirs(os.path.dirname(revenue_file), exist_ok=True)
        with open(revenue_file, 'w') as f:
            json.dump(self.revenue_data, f, indent=2)
    except Exception as e:
        print(f"❌ Revenue data save error: {e}")

def calculate_request_price(self, operation, is_premium=False, is_batch=False):
    """Calculate dynamic pricing for requests"""
    base_price = self.pricing_model.get(operation, {}).get("base", 0.01)

    # Premium pricing
    if is_premium:
        base_price = self.pricing_model.get(operation, {}).get("premium", base_price)

    # Peak hours multiplier
    current_hour = datetime.now().hour
    if 9 <= current_hour <= 17: # Business hours
        base_price *= self.pricing_model.get("peak_hours_multiplier", 1.2)

    # Batch discount
    if is_batch:
        discount = self.pricing_model.get("batch_processing", {}).get("discount", 0.1)
        base_price *= (1 - discount)

    return round(base_price, 4)

def process_revenue_event(self, operation, success=True, is_premium=False, is_batch=False):
    """Process revenue-generating event"""
    price = self.calculate_request_price(operation, is_premium, is_batch)

    self.revenue_data["total_requests"] += 1
    if success:
        self.revenue_data["successful_requests"] += 1
        self.revenue_data["daily_revenue"] += price
        self.revenue_data["monthly_revenue"] += price

    self.revenue_data["last_updated"] = datetime.now().isoformat()
    self._save_revenue_data()

    return {
        "price_charged": price,
        "total_daily_revenue": self.revenue_data["daily_revenue"],
        "success_rate": self.get_success_rate()
    }

def get_success_rate(self):
    """Calculate success rate percentage"""
    if self.revenue_data["total_requests"] == 0:
        return 0.0
    return (self.revenue_data["successful_requests"] /
            self.revenue_data["total_requests"]) * 100

def get_revenue_analytics(self):
    """Generate comprehensive revenue analytics"""
    current_date = datetime.now().date()
    start_date = datetime.fromisoformat(self.revenue_data["start_date"]).date()
    days_active = (current_date - start_date).days + 1

    # Calculate projections
    daily_avg = self.revenue_data["daily_revenue"] / max(days_active, 1)
    monthly_projection = daily_avg * 30
    annual_projection = daily_avg * 365

    # Target analysis
    daily_target = self.config["revenue_target_daily"]
    target_progress = (self.revenue_data["daily_revenue"] / daily_target) * 100

    return {
        "current_metrics": {
            "daily_revenue": self.revenue_data["daily_revenue"],
            "monthly_revenue": self.revenue_data["monthly_revenue"],
            "total_requests": self.revenue_data["total_requests"],
            "successful_requests": self.revenue_data["successful_requests"],
            "success_rate": self.get_success_rate()
        },
        "projections": {
            "monthly": monthly_projection,
            "annual": annual_projection
        },
        "target_analysis": {
            "daily_target": daily_target,
            "current_progress": target_progress
        }
    }

```



```

        "projections": {
            "daily_average": daily_avg,
            "monthly_projection": monthly_projection,
            "annual_projection": annual_projection
        },
        "targets": {
            "daily_target": daily_target,
            "progress_percent": target_progress,
            "target_gap": daily_target - self.revenue_data["daily_revenue"]
        },
        "performance": {
            "days_active": days_active,
            "avg_requests_per_day": self.revenue_data["total_requests"] / max(days_active, 1),
            "avg_revenue_per_request": (
                self.revenue_data["daily_revenue"] /
                max(self.revenue_data["successful_requests"], 1)
            )
        }
    }

def optimize_pricing(self):
    """Dynamic pricing optimization based on performance"""
    analytics = self.get_revenue_analytics()

    # If below target, suggest price increases
    if analytics["targets"]["progress_percent"] < 80:
        optimization_suggestions = {
            "action": "increase_pricing",
            "reason": "Below revenue target",
            "suggested_multiplier": 1.1,
            "focus_operations": ["text_generation", "image_processing"]
        }
    elif analytics["targets"]["progress_percent"] > 120:
        optimization_suggestions = {
            "action": "maintain_pricing",
            "reason": "Exceeding target",
            "suggested_strategy": "focus_on_volume"
        }
    else:
        optimization_suggestions = {
            "action": "optimize_operations",
            "reason": "On track",
            "suggested_focus": "efficiency_improvements"
        }

    return optimization_suggestions

def generate_business_report(self):
    """Generate comprehensive business report"""
    analytics = self.get_revenue_analytics()
    optimization = self.optimize_pricing()

    report = {
        "report_date": datetime.now().isoformat(),
        "revenue_summary": analytics["current_metrics"],
        "projections": analytics["projections"],
        "target_analysis": analytics["targets"],
        "performance_metrics": analytics["performance"],
        "optimization_recommendations": optimization,
        "pricing_model": self.pricing_model
    }

    return report

def export_analytics_data(self, format_type="json"):
    """Export analytics data for external analysis"""
    report = self.generate_business_report()

    if format_type == "json":
        filename = f"analytics_export_{datetime.now().strftime('%Y%m%d_%H%M%S')}.json"
        filepath = f"logs/{filename}"

        try:
            os.makedirs(os.path.dirname(filepath), exist_ok=True)
            with open(filepath, 'w') as f:
                json.dump(report, f, indent=2)
            return {"success": True, "file": filepath}
        except Exception as e:
            return {"success": False, "error": str(e)}

```

```

        return {"success": False, "error": "Unsupported format"}

def reset_daily_metrics(self):
    """Reset daily metrics (call this daily via cron)"""
    self.revenue_data["daily_revenue"] = 0.0
    self.revenue_data["last_updated"] = datetime.now().isoformat()
    self._save_revenue_data()
    print("🔄 Daily metrics reset")

def get_quick_stats(self):
    """Get quick business statistics"""
    return {
        "daily_revenue": f"${self.revenue_data['daily_revenue']:.2f}",
        "monthly_revenue": f"${self.revenue_data['monthly_revenue']:.2f}",
        "total_requests": self.revenue_data["total_requests"],
        "success_rate": f"{self.get_success_rate():.1f}%",
        "target_progress": f"{(self.revenue_data['daily_revenue'] / self.config['r
    }

def main():
    """Business controller CLI interface"""
    import sys

    controller = BusinessController()

    if len(sys.argv) < 2:
        print("OASIS v3 Business Controller")
        print("Available commands:")
        print("  stats    - Show quick statistics")
        print("  report   - Generate full business report")
        print("  optimize - Get pricing optimization suggestions")
        print("  export   - Export analytics data")
        print("  reset    - Reset daily metrics")
        return

    command = sys.argv[1]

    if command == "stats":
        stats = controller.get_quick_stats()
        print("📊 Quick Business Statistics:")
        for key, value in stats.items():
            print(f"  {key}: {value}")

    elif command == "report":
        report = controller.generate_business_report()
        print("📄 Business Report:")
        print(json.dumps(report, indent=2))

    elif command == "optimize":
        optimization = controller.optimize_pricing()
        print("🎯 Pricing Optimization:")
        print(json.dumps(optimization, indent=2))

    elif command == "export":
        result = controller.export_analytics_data()
        if result["success"]:
            print(f"✅ Analytics exported to: {result['file']}")
        else:
            print(f"❌ Export failed: {result['error']}")

    elif command == "reset":
        controller.reset_daily_metrics()

    else:
        print(f"❌ Unknown command: {command}")

if __name__ == "__main__":
    main()

```



6. Monitoring System

```
#!/usr/bin/env python3
"""
OASIS v3 Ultra-Lightweight Monitoring System
Real-time performance tracking and health checks
Pure Python implementation - Zero external dependencies
"""

import json
import time
import os
import sys
from datetime import datetime, timedelta
import urllib.request
import threading
import queue

class OasisMonitor:
    """Ultra-lightweight system monitoring"""

    def __init__(self, config_path="config/oasis_config.json"):
        self.config = self._load_config(config_path)
        self.metrics = {
            "system_health": "unknown",
            "api_response_time": [],
            "memory_usage": [],
            "request_counts": {"success": 0, "error": 0},
            "uptime_start": datetime.now(),
            "last_health_check": None,
            "alerts": []
        }
        self.monitoring_active = False
        self.alert_thresholds = {
            "response_time_ms": 5000, # 5 seconds
            "error_rate_percent": 10, # 10% error rate
            "memory_usage_mb": 400 # 400MB memory limit
        }

    def _load_config(self, config_path):
        """Load monitoring configuration"""
        default_config = {
            "monitoring_interval": 60,
            "health_check_url": "",
            "alert_enabled": True,
            "log_metrics": True
        }
        try:
            if os.path.exists(config_path):
                with open(config_path, 'r') as f:
                    config_data = json.load(f)
                    return config_data.get("monitoring", default_config)
            return default_config
        except Exception as e:
            print(f"⚠️ Monitor config load error: {e}")
            return default_config

    def check_system_health(self):
        """Perform comprehensive system health check"""
        health_report = {
            "timestamp": datetime.now().isoformat(),
            "status": "healthy",
            "checks": {}
        }

        # Memory usage check
        try:
            memory_info = self._get_memory_usage()
            health_report["checks"]["memory"] = {
                "status": "ok" if memory_info["usage_mb"] < self.alert_thresholds["memory_usage_mb"] else "error",
                "usage_mb": memory_info["usage_mb"],
                "available_mb": memory_info["available_mb"]
            }
        except Exception as e:
            health_report["checks"]["memory"] = {"status": "error", "error": str(e)}

        # API connectivity check
        if self.config.get("health_check_url"):
            api_health = self._check_api_connectivity()
            health_report["checks"]["api_connectivity"] = api_health
```

```

# File system check
storage_info = self._check_storage()
health_report["checks"]["storage"] = storage_info

# Performance metrics check
perf_check = self._check_performance_metrics()
health_report["checks"]["performance"] = perf_check

# Determine overall health
failed_checks = [check for check in health_report["checks"].values()
                  if check.get("status") in ["error", "critical"]]

if failed_checks:
    health_report["status"] = "unhealthy"
elif any(check.get("status") == "warning" for check in health_report["checks"]):
    health_report["status"] = "warning"

self.metrics["system_health"] = health_report["status"]
self.metrics["last_health_check"] = datetime.now()

return health_report

def _get_memory_usage(self):
    """Get current memory usage (simplified for Termux)"""
    try:
        # Basic memory info (works in most Unix-like systems)
        if os.path.exists("/proc/meminfo"):
            with open("/proc/meminfo", 'r') as f:
                meminfo = f.read()

            # Parse MemTotal and MemAvailable
            lines = meminfo.split('\n')
            mem_total = 0
            mem_available = 0

            for line in lines:
                if line.startswith("MemTotal:"):
                    mem_total = int(line.split()[1]) // 1024 # Convert to MB
                elif line.startswith("MemAvailable:"):
                    mem_available = int(line.split()[1]) // 1024 # Convert to MB

            usage_mb = mem_total - mem_available
            return {
                "total_mb": mem_total,
                "usage_mb": usage_mb,
                "available_mb": mem_available,
                "usage_percent": (usage_mb / mem_total) * 100 if mem_total > 0 else 0
            }
        except Exception as e:
            print(f"Memory check error: {e}")

        # Fallback estimation
        return {
            "total_mb": 1024, # Assume 1GB
            "usage_mb": 256, # Assume 256MB usage
            "available_mb": 768,
            "usage_percent": 25.0
        }

def _check_api_connectivity(self):
    """Check API endpoint connectivity"""
    url = self.config["health_check_url"]

    try:
        start_time = time.time()
        req = urllib.request.Request(url, headers={'User-Agent': 'OASIS-Monitor/1.0'})

        with urllib.request.urlopen(req, timeout=10) as response:
            response_time = (time.time() - start_time) * 1000 # Convert to ms

            self.metrics["api_response_time"].append(response_time)
            # Keep only last 10 measurements
            if len(self.metrics["api_response_time"]) > 10:
                self.metrics["api_response_time"] = self.metrics["api_response_time"][-10:]

            status = "ok" if response_time < self.alert_thresholds["response_time"] else "slow"

            return {
                "status": status,

```

```

        "response_time_ms": response_time,
        "http_status": response.status,
        "url": url
    }

except Exception as e:
    return {
        "status": "error",
        "error": str(e),
        "url": url
    }

def _check_storage(self):
    """Check available storage space"""
    try:
        # Get storage info for home directory
        statvfs = os.statvfs(os.path.expanduser("~"))

        # Calculate sizes in MB
        total_space = (statvfs.f_frsize * statvfs.f_blocks) // (1024 * 1024)
        free_space = (statvfs.f_frsize * statvfs.f_bavail) // (1024 * 1024)
        used_space = total_space - free_space
        usage_percent = (used_space / total_space) * 100 if total_space > 0 else 0

        status = "ok"
        if usage_percent > 90:
            status = "critical"
        elif usage_percent >

```